5.3 커스텀 브릿지 개발 가이드

개요

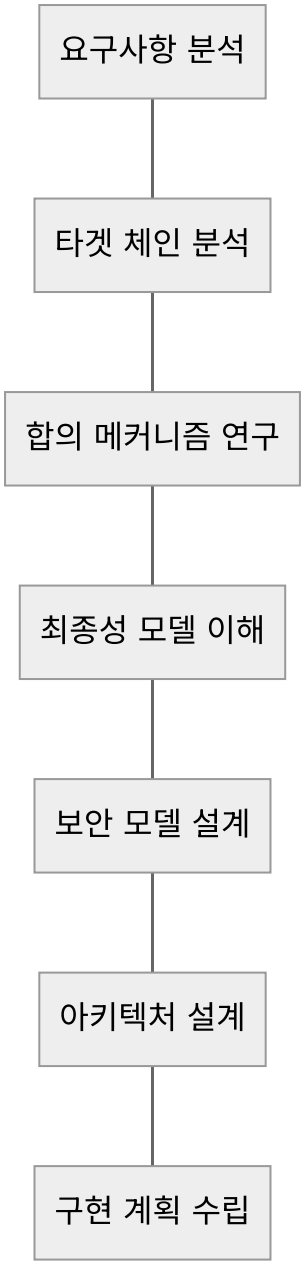
이 문서는 Polkadot SDK의 Bridges 프레임워크를 사용하여 새로운 블록체인과의 커스텀 브릿지를 개발하는 방법을 제공합니다. 기존의 GRANDPA 기반 Substrate 체인부터 완전히 다른 합의 메커니즘을 사용하는 외부 블록체인까지 다양한 시나리오를 다룹니다.

새로운 블록체인과의 브릿지 개발 방법론

5.3.1 브릿지 개발 단계별 접근법

1단계: 요구사항 분석 및 설계

브릿지 개발을 시작하기 전에 다음 요소들을 분석해야 합니다:



타겟 체인 특성 분석:

- 합의 알고리즘 (PoW, PoS, DPoS, BFT 등)
- 최종성 메커니즘 (확률적 vs 결정적)
- 블록 구조 및 헤더 포맷
- 서명 스킴 및 암호학적 프리미티브
- 네트워크 프로토콜 및 RPC 인터페이스

2단계: 라이트 클라이언트 설계

타겟 체인의 특성에 따라 적절한 라이트 클라이언트 전략을 선택합니다:

```rust
// 라이트 클라이언트 트레이트 정의 예시
pub trait LightClient<Header, Proof> {
type Error;
/// 헤더 검증
fn verify_header(&self, header: &Header, proof: &Proof) -> Result<(), Self::Err
/// 최종성 확인
fn is_finalized(&self, header: &Header) -> bool;
/// 상태 업데이트
fn update_state(&mut self, header: Header, proof: Proof) -> Result<(), Self::E
/// 스토리지 증명 검증
fn verify_storage_proof(
&self,
header: &Header,
proof: &StorageProof,
key: &[u8]
) -> Result<Option<Vec<u8>>, Self::Error>;
}
```

5.3.2 GRANDPA 기반 Substrate 체인 브릿지

기존 GRANDPA 팔렛 활용:

```rust
// Cargo.toml
```

```
[dependencies]

pallet-bridge-grandpa = { path = "../../modules/grandpa" }

bp-runtime = { path = "../../primitives/runtime" }

bp-header-chain = { path = "../../primitives/header-chain" }


// runtime/lib.rs

impl pallet_bridge_grandpa::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type BridgedChain = bp_my_chain::MyChain;

type MaxRequests = MaxRequests;

type HeadersToKeep = HeadersToKeep;

type WeightInfo = pallet_bridge_grandpa::weights::BridgeWeight<Runtime>;

}
```

체인별 프리미티브 정의:

```
// primitives/my-chain/src/lib.rs

use bp_runtime::Chain;

use frame_support::weights::Weight;


pub  struct MyChain;


impl Chain for MyChain {

const ID: ChainId = *b"mych";
```

```
type BlockNumber = u32;

type Hash = sp_core::H256;

type Hasher = sp_runtime::traits::BlakeTwo256;

type Header = sp_runtime::generic::Header<Self::BlockNumber, Self::Hasher>;

type AccountId = sp_runtime::AccountId32;

type Balance = u128;

type Nonce = u32;

type Signature = sp_runtime::MultiSignature;

fn  max_extrinsic_size() -> u32 {

4 * 1024 * 1024  // 4MB

}

fn  max_extrinsic_weight() -> Weight {

Weight::from_parts(2_000_000_000_000, 64 * 1024)

}

}
```

5.3.3 비-GRANDPA 체인 브릿지 개발

커스텀 라이트 클라이언트 팔렛 구현:

```
// pallets/my-chain-light-client/src/lib.rs

#[frame_support::pallet]

pub  mod pallet {

use  super::*;
```

```
use frame_support::pallet_prelude::*;

use frame_system::pallet_prelude::*;


#[pallet::pallet]

pub struct Pallet<T>(_);


#[pallet::config]

pub trait Config: frame_system::Config {

type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::

type BridgedChain: bp_runtime::Chain;

type WeightInfo: WeightInfo;

}


#[pallet::storage]

pub type BestFinalized<T: Config> = StorageValue<

_,

BridgedHeader<T>,

OptionQuery,

>;


#[pallet::storage]

pub type ImportedHeaders<T: Config> = StorageMap<

_,
```

```
Blake2_128Concat,
BridgedBlockHash<T>,
BridgedHeader<T>,
OptionQuery,
>;


#[pallet::call]
impl<T: Config> Pallet<T> {
/// 커스텀 합의 증명과 함께 헤더 제출
#[pallet::call_index(0)]
#[pallet::weight(T::WeightInfo::submit_finality_proof())]
pub fn submit_finality_proof(
origin: OriginFor<T>,
finality_target: BridgedHeader<T>,
justification: MyChainJustification,
) -> DispatchResult {
ensure_signed(origin)?;
// 커스텀 검증 로직 구현
Self::verify_justification(&finality_target, &justification)?;
// 헤더 저장
Self::insert_header(finality_target)?;
Ok(())
}
```

```
}

impl<T: Config> Pallet<T> {
/// 커스텀 합의 메커니즘에 따른 정당화 검증
fn verify_justification(
header: &BridgedHeader<T>,
justification: &MyChainJustification,
) -> Result<(), Error<T>> {
// 타겟 체인의 합의 규칙에 따른 검증 로직
// 예: PoW 난이도 검증, PoS 서명 검증 등
todo!("Implement custom consensus verification")
}
}
}
```

5.3.4 이더리움 스타일 PoW 체인 브릿지

PoW 검증 로직 구현:

```
// 이더리움 스타일 PoW 검증 예시
pub struct EthereumPoWVerifier;

impl EthereumPoWVerifier {
```

```rust
pub fn verify_pow_header(
header: &EthereumHeader,
parent: &EthereumHeader,
) -> Result<(), PoWError> {
// 1. 블록 번호 검증
ensure!(
header.number == parent.number + 1,
PoWError::InvalidBlockNumber
);
// 2. 타임스탬프 검증
ensure!(
header.timestamp > parent.timestamp,
PoWError::InvalidTimestamp
);
// 3. 난이도 조정 검증
let expected_difficulty = Self::calculate_difficulty(parent, header.timestamp);
ensure!(
header.difficulty == expected_difficulty,
PoWError::InvalidDifficulty
);
// 4. PoW 해시 검증
let pow_hash = Self::calculate_pow_hash(header);
ensure!(
```

```
pow_hash <= Self::difficulty_to_target(header.difficulty),
PoWError::InvalidPoW
);
Ok(())
}
fn calculate_difficulty(parent: &EthereumHeader, timestamp: u64) -> U256 {
// 이더리움 난이도 조정 알고리즘 구현
todo!()
}
fn calculate_pow_hash(header: &EthereumHeader) -> H256 {
// Ethash 알고리즘 구현
todo!()
}
}
```

브릿지 팔렛 커스터마이징 및 확장 방법

5.3.5 메시지 팔렛 커스터마이징

커스텀 메시지 디스패처 구현:

```
// 커스텀 메시지 디스패처
pub struct MyChainMessageDispatch<T>(PhantomData<T>);
```

```
impl<T: Config> MessageDispatch for MyChainMessageDispatch<T> {
type DispatchPayload = Vec<u8>;
type DispatchLevelResult = ();
fn  is_active() -> bool {
// 브릿지 활성 상태 확인
true
}
fn  dispatch_weight(payload: &Self::DispatchPayload) -> Weight {
// 메시지 실행 가중치 계산
Weight::from_parts(1_000_000, 0)
}
fn  dispatch(
relayer_account: &T::AccountId,
payload: Self::DispatchPayload,
) -> MessageDispatchResult<Self::DispatchLevelResult> {
// 커스텀 메시지 처리 로직
match  Self::decode_and_execute_message(payload) {
Ok(result) => MessageDispatchResult {
unspent_weight: Weight::zero(),
dispatch_level_result: result,
},
Err(e) => MessageDispatchResult {
unspent_weight: Weight::zero(),
```

```
dispatch_level_result: (),
},
}
}
}
```

커스텀 메시지 페이로드 정의:

```
// 커스텀 메시지 타입
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub  enum MyChainMessage {
/// 토큰 전송
Transfer {
recipient: AccountId32,
amount: u128,
token_id: H256,
},
/// 스마트 컨트랙트 호출
ContractCall {
contract: H160,
data: Vec<u8>,
value: u128,
},
/// 거버넌스 제안
```

```
GovernanceProposal {
proposal_hash: H256,
voting_period: u32,
},
}

impl MyChainMessage {
pub fn encode_for_bridge(&self) -> Vec<u8> {
self.encode()
}
pub fn decode_from_bridge(data: &[u8]) -> Result<Self, codec::Error> {
Self::decode(&mut &data[..])
}
}
```

5.3.6 릴레이어 커스터마이징

커스텀 릴레이어 구현:

```
// 커스텀 릴레이어 서비스
pub struct MyChainRelay<SC, TC> {
source_client: SC,
target_client: TC,
metrics: Option<MetricsParams>,
```

```
}

impl<SC, TC> MyChainRelay<SC, TC>
where
SC: SourceClient,
TC: TargetClient,
{
pub  async  fn  run(self) -> Result<(), RelayError> {
let  mut  source_subscription = self.source_client.subscribe_finalized_headers()
while  let Some(header) = source_subscription.next().await {
match  self.process_header(header).await {
Ok(_) => log::info!("Successfully processed header"),
Err(e) => log::error!("Failed to process header: {:?}", e),
}
}
Ok(())
}
async  fn  process_header(&self, header: SourceHeader) -> Result<(), RelayError>
// 1. 헤더 검증
self.verify_header(&header)?;
// 2. 필요한 증명 생성
let  proof = self.generate_finality_proof(&header).await?;
// 3. 타겟 체인에 제출
```

```
let  call = self.build_submit_call(header, proof);

self.target_client.submit_signed_extrinsic(call).await?;

Ok(())

}

fn  generate_finality_proof(&self, header: &SourceHeader) -> Result<FinalityProo

// 소스 체인의 합의 메커니즘에 따른 증명 생성

todo!()

}

}
```

보안 고려사항 및 감사 체크리스트

5.3.7 보안 위험 요소 분석

1. 합의 메커니즘 보안

```
// 보안 검증 체크리스트

pub  struct SecurityAudit;



impl SecurityAudit {

/// 라이트 클라이언트 보안 검증

pub  fn  audit_light_client<T: Config>() -> AuditResult {

let  mut  issues = Vec::new();

// 1. 초기화 보안 검증
```

```
if !Self::check_initialization_security::<T>() {
issues.push(SecurityIssue::UnsafeInitialization);
}
// 2. 합의 검증 로직 감사
if !Self::audit_consensus_verification::<T>() {
issues.push(SecurityIssue::WeakConsensusVerification);
}
// 3. 상태 전환 보안 검증
if !Self::check_state_transition_security::<T>() {
issues.push(SecurityIssue::UnsafeStateTransition);
}
// 4. 재생 공격 방지 검증
if !Self::check_replay_protection::<T>() {
issues.push(SecurityIssue::ReplayVulnerability);
}
AuditResult { issues }
}
fn  check_initialization_security<T: Config>() -> bool {
// 초기 상태 설정의 보안성 검증
// - 신뢰할 수 있는 체크포인트 사용
// - 적절한 권한 검증
// - 초기화 후 변경 불가능성
true
```

```
}
fn audit_consensus_verification<T: Config>() -> bool {
// 합의 검증 로직의 보안성 감사
// - 서명 검증 강도
// - 임계값 설정 적절성
// - 암호학적 프리미티브 안전성
true
}
}

#[derive(Debug)]
pub enum SecurityIssue {
UnsafeInitialization,
WeakConsensusVerification,
UnsafeStateTransition,
ReplayVulnerability,
InsufficientValidation,
EconomicAttackVector,
}
```

2. 경제적 공격 벡터

```
// 경제적 보안 분석
```

```
pub struct EconomicSecurity;


impl EconomicSecurity {
/// 릴레이어 인센티브 분석
pub fn analyze_relayer_incentives(
relay_cost: Balance,
reward: Balance,
success_probability: f64,
) -> IncentiveAnalysis {
let expected_profit = (reward as f64 * success_probability) - relay_cost as f
IncentiveAnalysis {
expected_profit,
is_profitable: expected_profit > 0.0,
risk_level: Self::calculate_risk_level(success_probability),
}
}
/// 그리핑 공격 저항성 검증
pub fn check_griefing_resistance<T: Config>() -> bool {
// 1. 최소 보상 요구사항 확인
let min_reward = T::MinRelayerReward::get();
// 2. 스팸 방지 메커니즘 확인
let has_spam_protection = T::SpamProtection::enabled();
// 3. 경제적 처벌 메커니즘 확인
```

```
let  has_slashing = T::SlashingEnabled::get();

min_reward > 0 && has_spam_protection && has_slashing

}

}
```

5.3.8 감사 체크리스트

코드 품질 검증:

```
## 브릿지 보안 감사 체크리스트


### 1. 아키텍처 보안

- [ ] 라이트 클라이언트 설계 검토

- [ ] 신뢰 가정 명확화

- [ ] 공격 벡터 분석 완료

- [ ] 장애 모드 분석 완료


### 2. 구현 보안

- [ ] 입력 검증 완전성

- [ ] 정수 오버플로우 방지

- [ ] 메모리 안전성 보장

- [ ] 패닉 상황 처리


### 3. 암호학적 보안
```

- [ ] 서명 스킴 검증
- [ ] 해시 함수 안전성
- [ ] 난수 생성 품질
- [ ] 키 관리 보안

4. 경제적 보안

- [ ] 인센티브 정렬 검증
- [ ] 공격 비용 분석
- [ ] 보상 메커니즘 검토
- [ ] 슬래싱 조건 검증

5. 운영 보안

- [ ] 업그레이드 메커니즘
- [ ] 비상 중단 절차
- [ ] 모니터링 시스템
- [ ] 사고 대응 계획

브릿지 테스트 및 검증 방법론

5.3.9 단위 테스트 프레임워크

팔렛 테스트 구조:

```
// tests/mock.rs
```

```
use frame_support::{
construct_runtime, parameter_types,
traits::{ConstU32, ConstU64},
};

construct_runtime!(
pub  struct TestRuntime {
System: frame_system,
MyChainBridge: pallet_my_chain_bridge,
Messages: pallet_bridge_messages,
}
);

parameter_types! {
pub  const BlockHashCount: u64 = 250;
}

impl frame_system::Config for TestRuntime {
type BaseCallFilter = frame_support::traits::Everything;
type BlockWeights = ();
type BlockLength = ();
type DbWeight = ();
type RuntimeOrigin = RuntimeOrigin;
```

```
type RuntimeCall = RuntimeCall;

type Nonce = u64;

type Hash = H256;

type Hashing = BlakeTwo256;

type AccountId = u64;

type Lookup = IdentityLookup<Self::AccountId>;

type Block = Block;

type RuntimeEvent = RuntimeEvent;

type BlockHashCount = BlockHashCount;

type Version = ();

type PalletInfo = PalletInfo;

type AccountData = ();

type OnNewAccount = ();

type OnKilledAccount = ();

type SystemWeightInfo = ();

type SS58Prefix = ();

type OnSetCode = ();

type MaxConsumers = ConstU32<16>;

}



// tests/lib.rs

#[cfg(test)]

mod tests {
```

```rust
use super::*;
use frame_support::{assert_ok, assert_noop};
#[test]
fn submit_valid_header_works() {
new_test_ext().execute_with(|| {
// 유효한 헤더와 증명 생성
let header = create_valid_header();
let proof = create_valid_proof(&header);
// 헤더 제출 테스트
assert_ok!(MyChainBridge::submit_finality_proof(
RuntimeOrigin::signed(1),
header.clone(),
proof
));
// 상태 검증
assert_eq!(MyChainBridge::best_finalized(), Some(header));
});
}
#[test]
fn submit_invalid_proof_fails() {
new_test_ext().execute_with(|| {
let header = create_valid_header();
let invalid_proof = create_invalid_proof();
```

```
assert_noop!(
MyChainBridge::submit_finality_proof(
RuntimeOrigin::signed(1),
header,
invalid_proof
),
Error::<TestRuntime>::InvalidFinalityProof
);
});
}
}
```

5.3.10 통합 테스트

Zombienet 기반 통합 테스트:

```
# zombienet-config.toml
[settings]
timeout = 1000

[relaychain]
default_image = "parity/polkadot:latest"
default_command = "polkadot"
default_args = ["--tmp"]
```

```
chain = "rococo-local"


[[relaychain.nodes]]

name = "alice"

validator = true


[[relaychain.nodes]]

name = "bob"

validator = true


[[parachains]]

id = 1000

cumulus_based = true


[[parachains.collators]]

name = "bridge-hub-rococo"

image = "parity/polkadot-parachain:latest"

command = "polkadot-parachain"


[[parachains]]

id = 2000

cumulus_based = true
```

```
[[parachains.collators]]
name = "my-chain-bridge"
image = "my-org/my-chain-bridge:latest"
command = "my-chain-bridge"
```

테스트 시나리오 스크립트:

```bash
#!/bin/bash
# test-bridge-functionality.sh

set -e

echo "Starting bridge integration test..."

# 1. 네트워크 시작
zombienet spawn zombienet-config.toml &
ZOMBIENET_PID=$!

sleep 60 # 네트워크 초기화 대기

# 2. 브릿지 초기화
echo "Initializing bridge..."
polkadot-js-api --ws ws://localhost:9944 \
```

```
tx.myChainBridge.initialize \
--seed "//Alice" \
--params initial_header.json


# 3. 메시지 전송 테스트
echo "Testing message transfer..."
polkadot-js-api --ws ws://localhost:9944 \
tx.bridgeMessages.sendMessage \
--seed "//Bob" \
--params test_message.json


# 4. 릴레이어 시작
echo "Starting relayer..."
./target/release/my-chain-relay \
--source-host localhost \
--source-port 8545 \
--target-host localhost \
--target-port 9944 &
RELAY_PID=$!


# 5. 메시지 전달 확인
sleep 30
echo "Verifying message delivery..."
```

```
# 메시지 전달 상태 확인 로직


# 정리
kill $RELAY_PID
kill $ZOMBIENET_PID


echo "Integration test completed successfully!"
```

5.3.11 성능 및 부하 테스트

벤치마킹 구현:

```
// benches/bridge_benchmarks.rs
use criterion::{black_box, criterion_group, criterion_main, Criterion};
use pallet_my_chain_bridge::*;


fn benchmark_header_verification(c: &mut Criterion) {
let header = create_test_header();
let proof = create_test_proof();
c.bench_function("verify_header", |b| {
b.iter(|| {
verify_finality_proof(
black_box(&header),
black_box(&proof)
```

```
)
})
});
}

fn benchmark_message_processing(c: &mut Criterion) {
let messages = create_test_messages(100);
c.bench_function("process_messages", |b| {
b.iter(|| {
for message in &messages {
process_inbound_message(black_box(message));
}
})
});
}

criterion_group!(benches, benchmark_header_verification, benchmark_message_proce
criterion_main!(benches);
```

5.3.12 보안 테스트

퍼징 테스트:

```
// fuzz/fuzz_targets/header_verification.rs
```

```rust
#![no_main]

use libfuzzer_sys::fuzz_target;

use pallet_my_chain_bridge::*;



fuzz_target!(|data: &[u8]| {

if  data.len() < 100 {

return;

}

// 임의의 데이터로 헤더 생성 시도

if  let Ok(header) = decode_header(&data[..50]) {

if  let Ok(proof) = decode_proof(&data[50..]) {

// 검증 함수가 패닉하지 않는지 확인

let  _ = verify_finality_proof(&header, &proof);

}

}

});
```

속성 기반 테스트:

```rust
// tests/property_tests.rs

use proptest::prelude::*;



proptest! {
```

```
#[test]
fn valid_headers_always_verify(
block_number in 1u32..1000000,
parent_hash in any::<[u8; 32]>(),
state_root in any::<[u8; 32]>()
) {
let header = create_header_with_params(block_number, parent_hash, state_root);
let proof = create_valid_proof_for_header(&header);
prop_assert!(verify_finality_proof(&header, &proof).is_ok());
}
#[test]
fn invalid_proofs_always_fail(
header in any_valid_header(),
corrupted_proof in any_corrupted_proof()
) {
prop_assert!(verify_finality_proof(&header, &corrupted_proof).is_err());
}
}
```

모범 사례 및 권장사항

5.3.13 개발 모범 사례

1. 점진적 개발 접근법:

- 기존 GRANDPA 브릿지를 참조하여 시작
- 최소 기능으로 시작하여 점진적 확장
- 각 단계별 철저한 테스트

2. 보안 우선 설계:

- 보안 검토를 개발 초기부터 통합
- 외부 보안 감사 수행
- 버그 바운티 프로그램 운영

3. 모듈화 및 재사용성:

- 공통 컴포넌트 추상화
- 다른 브릿지에서 재사용 가능한 설계
- 명확한 인터페이스 정의

이 문서는 커스텀 브릿지 개발의 전체 과정을 다뤘습니다. 실제 구현 시에는 타겟 체인의 특성에 맞게 세부사항을 조정해야 하며, 보안과 성능을 항상 최우선으로 고려해야 합니다.